পরীক্ষা 1-এর সংশোধন: if, if-else এবং শর্ত

পাঠ 11 — 1IOT, SST-ICT-01 — if গঠনের পুনরালোচনা, শর্ত, ছাড়ের অনুশীলন, বন্ধনী, একাধিক লাইনে লেখা, Python-এর সাথে তুলনা

13/10/2026

1 আমরা কোথা থেকে আবার শুরু করি

টীকা

এই পাঠ এসেছে 12/10-এর **পরীক্ষা 1-এর সংশোধন** (**verifica** = পরীক্ষা) থেকে। দুটি বিষয়ে বেশি অসুবিধা হয়েছে, সেগুলো আবার দেখতে হবে:

- বাইনারিতে **সংমিশ্রণের সংখ্যার** সূত্র;
- `if` **গঠন**, বিশেষ করে **শর্তে** কী লেখা হয়।

শিক্ষক স্পষ্ট করে বলেন: `if` ব্যবহার হয় "**সকাল থেকে সন্ধ্যা**, প্রতিদিন"। এটি না বুঝলে পরের **যেকোনো বিষয়** আটকে যায়। তাই এগোনোর আগে এক ধাপ পিছিয়ে সব গুছিয়ে নেওয়া হয়। এসব আগের পাঠ্যেই আছে (A08 ও A09), কিন্তু **আবার দেখা** দরকার, কারণ পরীক্ষায় অনেকে ভুল করেছে।

2 সংমিশ্রণের সংখ্যা: 2^n

সংজ্ঞা

nটি **বাতি** (অর্থাৎ n বিট) দিয়ে, প্রতিটি জ্বলা বা নেভানো, সম্ভাব্য **সংমিশ্রণের সংখ্যা** হলো

$$\text{সংমিশ্রণ} = 2^n$$

যেমন: 1টি বাতি দেয় $2^1 = 2$টি সংমিশ্রণ, 3টি বাতি দেয় $2^3 = 8$টি, 8টি বাতি (এক বাইট) দেয় $2^8 = 256$টি। এই সূত্র খাতায় **দাগ দিয়ে রাখতে** হবে: পরীক্ষায় এটিতেই সবচেয়ে বেশি ভুল হয়েছে।

3 if গঠন: করা বা না করা

`if` গঠন একটি **সিদ্ধান্তের গঠন**: এটি যন্ত্রকে কোডের একটি অংশ চালাবে কিনা তা **বেছে নিতে** দেয়। দুটি গঠন আছে, পরস্পরের **বোনের** মতো।

সংজ্ঞা

`if` গঠন। **গোল** **বন্ধনীর** ভেতরে একটি **শর্ত** লেখা হয়; **কার্লি** **বন্ধনীর** ভেতরে চালানোর কোড। বোর্ডে শিক্ষক তিনটি রং ব্যবহার করেন: শর্ত **লাল**, `if`-এর কোড **সবুজ**।

```
if (condizione)
{
    printf("...");      // ভেতরে যেকোনো কোড থাকতে পারে:
    z = 3 + 5;          // ছাপা, হিসাব, আরও অনেক কিছু
}
```

সবুজ অংশ চলে **শুধু** **তখনই**, যখন লাল শর্ত **সত্য** হিসেবে মূল্যায়িত হয়। মিথ্যা হলে প্রোগ্রাম সেটি বাদ দিয়ে এগোয়।

সংজ্ঞা

`if-else` গঠন। একই জিনিস, সাথে একটি দ্বিতীয় ব্লক:

```
if (condizione)
{
    // "সবুজ" কোড: শর্ত সত্য হলে চলে
}
else
{
    // "নীল" কোড: শর্ত মিথ্যা হলে চলে
}
```

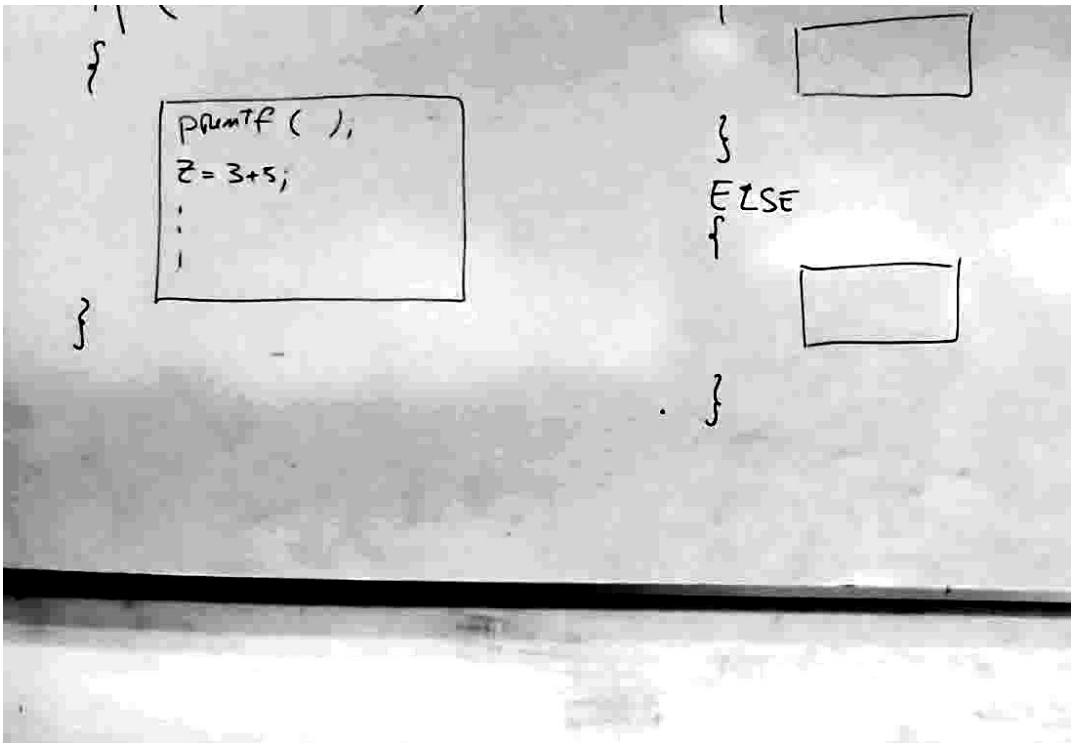
উদাহরণ

কোনটি বাছব? দুটি গঠন খুব কাছাকাছি, দুটি আলাদা জিনিস হিসেবে শেখার দরকার নেই: কী করতে হবে তা দেখে একটি বা অন্যটি ব্যবহার হয়।

- কিছু **করতে** **বা** **না** **করতে** হবে? `if`-ই যথেষ্ট।
- দুটি আলাদা কাজের মধ্যে **একটি** **বাছতে** হবে? `if-else` লাগবে।

সতর্কতা

সত্য ও মিথ্যা, ইতিবাচক ও নেতিবাচক নয়। ক্লাসে আসা একটি সন্দেহ: "পরীক্ষায় আমি **ইতিবাচক** আর **নেতিবাচক** দিয়ে ভেবে গুলিয়ে ফেলেছিলাম"। ইতিবাচক ও নেতিবাচক শব্দ ফ্লোচার্টে চলার একটি **দিক** মনে করায়। শর্তে কিন্তু শুধু **সত্য** বা **মিথ্যার** কথা হয়: এই **ধারণাগত** **ধাপটি** ঠিক করে নিতে হবে।



ছবি/নকশা 1: পাঠের শুরুতে বোর্ড: বাঁয়ে `if` গঠন, ব্লকে কোডের একটি উদাহরণসহ (একটি `printf` ও একটি হিসাব), ডানে দুটি ব্লকসহ `if-else` গঠন।

4 শর্তে কী লেখা হয়

সংজ্ঞা

শর্ত হলো যেকোনো **প্রকাশ** (**espressione**), যার ফলাফল **সত্য** বা **মিথ্যা**। সবচেয়ে সাধারণ হলো দুটি মানের **তুলনা**, আগে দেখা তুলনার অপারেটর দিয়ে (`==`, `!=`, `>`, `>=`, `<`, `<=`)।

উদাহরণ

বোর্ডের উদাহরণ।

- `a == b`: দুটি ভেরিয়েবলের **সমতা**। `a`-তে 5 আর `b`-তে 5 থাকলে ফলাফল **সত্য**; `a` 5 আর `b` 7 হলে **মিথ্যা**।
- `a > b`: `a` 6 আর `b` 5 হলে স্পষ্টতই সত্য, কারণ 6, 5-এর চেয়ে বড়। কিন্তু `a` 5 আর `b`-ও 5 হলে? **মিথ্যা**: 5, 5-এর চেয়ে বড় নয়।
- `a >= b`: দুটি মান সমান হলেও **সত্য** পেতে **বড়** বা **সমান** ব্যবহার হয়, যা সমান হওয়ার সম্ভাবনাকেও "ঝুড়িতে রাখে"। `a = 5` আর `b = 5` হলে সত্য।

একটি সীমার ক্ষেত্র: `b = 5` আর `a = 5,0000001` হলে সংখ্যা দুটি **আলাদা**, তাই `a > b` সত্য (এবং অবশ্যই `a >= b`-ও)।

টীকা

বড় ও বড় বা সমানের পাশে একইভাবে আছে **ছোট** (<) আর **ছোট বা সমান** (<=)। উচ্চ গণিত লাগে না, তবে **বড় ও ছোট** ভালোভাবে জানতে হবে: এটি একটি সমস্যা, কিন্তু **সমাধানযোগ্য**।

5 পরীক্ষার অনুশীলন: ছাড়

উদাহরণ

প্রশ্ন। একজন ব্যবহারকারী **ছাড় (sconto) পাবে** যদি তার বয়স **12 বছরের কম অথবা 65 বছর বা তার বেশি** হয়।

ধাপ 1: বাক্যটি ভাঙা

বাক্যটিকে তার প্রতীকে ভাঙা হয়। **দুটি শর্ত** আছে:

- বয়স **12-এর ছোট**: `eta < 12`;
- বয়স **65-এর বড় বা সমান**: `eta >= 65` (বোর্ডে "বড়"-র নিচে শিক্ষক "সমান" যোগ করেন)।

কোডে ভেরিয়েবলের নাম `eta`, **অ্যাকসেন্ট ছাড়া**: বোর্ডে লেখা "età" (বয়স), কিন্তু ভেরিয়েবলের নামে অ্যাকসেন্ট চলে না (A09)। দুটি শর্ত **"অথবা"** (**oppure**) শব্দে জোড়া, অর্থাৎ **OR** ধরনের যুক্তি।

সতর্কতা

OR কীভাবে লেখা হয়। OR লেখা হয় **দুটি খাড়া দাগ** দিয়ে: `||`। এটি slash (`/`) **নয়** এবং একটি দাগও নয়। খাড়া দাগকে ইংরেজিতে বলে **pipe**: Unix ব্যবস্থাতেও এই নাম ব্যবহার হয়, যেখানে এটি নির্দেশগুলোর বিভাজক। ক্লাসে শিক্ষক নামটি নিয়ে পুরো নিশ্চিত ছিলেন না: নামটি ঠিক।

ধাপ 2: সম্পূর্ণ শর্ত

```
if ((eta < 12) || (eta >= 65))
{
    printf("Sconto");
}
else
{
    printf("No sconto");
}
```

সংজ্ঞা

প্রতিটি শর্তের চারপাশে গোল বন্ধনী কেন। শিক্ষক প্রতিটি শর্ত **গোল বন্ধনীতে** রাখতে পছন্দ করেন: এতে প্রকাশটি **"সুরক্ষিত"** থাকে এবং অপারেটরগুলো কোন ক্রমে মূল্যায়িত হয় তা নিয়ে ভুল বোঝার সুযোগ থাকে না।

সত্যি বলতে এই অনুশীলনে `if (eta < 12 || eta >= 65)` লিখলেও চলত: প্রকাশটি পড়ার অন্য কোনো অর্থপূর্ণ উপায় নেই। তবু বন্ধনী দেওয়া হয়, কারণ পাঠক সঙ্গে সঙ্গে বোঝে এবং ভুলের ঝুঁকি থাকে না।

সতর্কতা

বন্ধনী ও বহনযোগ্যতা। ক্লাসে শিক্ষক বন্ধনীর কারণ হিসেবে এটিও বলেছিলেন যে "কিছু কম্পাইলার অগ্রাধিকার বদলায়"। স্পষ্টীকরণ: C-তে **অপারেটরের অগ্রাধিকার** ভাষার **মান দিয়ে স্থির** (A10-এও বলা হয়েছে), তাই একটি সঠিক কম্পাইলার তা বদলায় না। **বহনযোগ্যতা** (**portabilità**) নিয়ে সাধারণ কথাটি সত্য থাকে: অন্য যন্ত্রে (PC, Mac, Linux, Arduino) কম্পাইল করা একই কোড অন্য কম্পাইলার ও অন্য প্রসেসর ব্যবহার করে, এবং কিছু জিনিস বদলাতে পারে (যেমন `int`-এর আকার)। ব্যবহারিক নিয়ম বদলায় না: যৌগিক শর্তে **সবসময় বন্ধনী**।

আরেকটি সমাধান: দুটি if

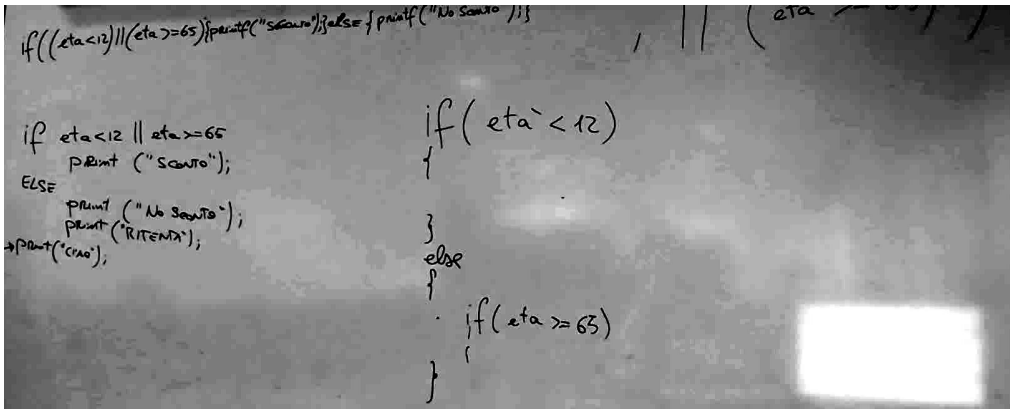
পরীক্ষায় কেউ কেউ **দুটি** `if` দিয়ে সমাধান করেছে: প্রথমে দেখে বয়স 12-এর ছোট কিনা; না হলে, `else`-এ, দেখে অন্তত 65 কিনা।

```
if (eta < 12)
{
    printf("Sconto");
}
else
{
    if (eta >= 65)
    {
        printf("Sconto");
    }
    else
    {
        printf("No sconto");
    }
}
```

এটি কাজ করে: বয়স হয় 12-এর ছোট, নয় অন্তত 65, আর দুটো একসাথে কখনো হতে পারে না। শিক্ষক এটিকে **ভালো সমাধান** মনে করেন। তবে OR-এর সমাধান বেশি **সংক্ষিপ্ত**: একটি `if`, একটি শর্ত। ("No sconto" সহ ভেতরের `else` ব্লকটি পুরো প্রোগ্রাম দেখাতে এখানে সম্পূর্ণ করা হয়েছে; বোর্ডে এটি খসড়া ছিল।)

6 শর্ত একাধিক লাইনে লেখা

ক্লাসের প্রশ্ন: সবসময় কি সব ভেঙে বন্ধনী দিতে হবে? আর লম্বা শর্তের ভেতরে কি নতুন লাইনে যাওয়া যায়?



ছবি/নকশা 2: পাঠের শেষে বোর্ড। ওপরে বাঁয়ে: ছাড়ের পুরো `if-else` এক লাইনে লেখা। নিচে: একই যুক্তি "Python-এর মতো", বন্ধনী ছাড়া, ইন্ডেন্টেশনসহ। ডানে: দুটি `if`-এর সমাধান, দ্বিতীয়টি `else`-এর ভেতরে (যে মানটি "63" মনে হয় সেটি 65 পড়তে হবে)।

- C-র কাছে **ফাঁক** আর নতুন লাইনের **গুরুত্ব** **নেই**: শর্ত একাধিক লাইনে ভাঙা যায়, প্রকাশ একই থাকে।
- তবে এলোমেলো ট্যাব দিয়ে এটি করা **হয়** **না**: বাকি কোডের **ইন্ডেন্টেশন** (**indentazione**) নষ্ট হয়।

```
if (
    (eta < 12)
    ||
    (eta >= 65)
   )
```

টীকা

দুই বা তিনটি শর্ত এভাবে লেখা **বাড়াবাড়ি**: এক লাইনই যথেষ্ট। শর্ত **অনেক** হলে এটি ভাবার অর্থ হয়। যেমন: ছাড় যদি বয়স 12-এর কম, অথবা 65-এর বেশি, অথবা 42, অথবা 47, অথবা 32, অথবা চুল সোনালি হয়... এসব ক্ষেত্রে সাধারণত **শর্তগুলো** অন্য জায়গায় **জড়ো** **করা** হয় এবং কোডে শুধু একটি **নাম** (লেবেল) ব্যবহার হয় যা সেগুলো বোঝায়: এই বিষয়টি আমরা পরে দেখব।

সতর্কতা

সব এক লাইনে করা যায়, কিন্তু করা হয় না। C-র কাছে নতুন লাইনের গুরুত্ব নেই বলে পুরো `if-else` **এক** **লাইনে** লেখা যেত:

```
if((eta<12)||(eta>=65)){printf("Sconto");}else{printf("No sconto");}
```

কম্পাইলার এটি মেনে নেয়, কিন্তু কেউ এভাবে করে না: **এটি** **পড়া** **যায়** **না**, আর ভুল হলে কম্পাইলার লাইন দেখায়, যা এখানে একটিই, তাই ভুল খোঁজা কঠিন হয়। ওপরের কোডগুলোর মতো **ইন্ডেন্ট** **করে** লিখতে হয়।

7 Python-এর সাথে তুলনা: বন্ধনী না ইন্ডেন্টেশন

Python নিয়ে একটি প্রশ্ন। শিক্ষক সততার সাথে আগেই বলেন যে Python তিনি সাধারণত ব্যবহার করেন

না; এখানে দরকারি পার্থক্য একটিই।

- **C**-তে `if`-এর ব্লকের সীমা চিহ্নিত করে **কার্লি বন্ধনী**।
- **Python**-এ বন্ধনী নেই: **ইন্ডেন্টেশনই** বলে কোন কোড `if` বা `else`-এর অংশ।

```
if eta < 12 or eta >= 65:
    print("Sconto")
else:
    print("No sconto")
    print("Ritenta")
print("Ciao")
```

`else`-এর নিচে ইন্ডেন্ট করা দুটি নির্দেশ ("No sconto" আর "Ritenta") চলে শুধু শর্ত মিথ্যা হলে, যেন বন্ধনীর ভেতরে আছে। `if`-এর সাথে সারিবদ্ধ `print("Ciao")`-এর `if`-এর সাথে **কোনো সম্পর্ক নেই**: এটি **সব ক্ষেত্রেই** চলে। শিক্ষকের মতে এটি একটু বেশি গোলমেলে: **বন্ধনী ব্লকের সীমা** স্পষ্ট দেখায়, Python-এ শুধু ইন্ডেন্টেশন তা করে।

টীকা

সংশোধনের টীকা। বোর্ডে Python-এর উদাহরণটি ছকের মতো লেখা ছিল, || আর সেমিকোলনসহ। আসল Python-এ OR লেখা হয় `or` শব্দ দিয়ে, শর্তের পরে ও `else`-এর পরে **কোলন** দিতে হয়, আর শেষের সেমিকোলন লাগে না। ওপরের কোড সঠিক রূপে লেখা।

8 আমরা কীভাবে এগোব

টীকা

- আমরা **C** নিয়েই চলব: মৌলিক গঠন ও উদাহরণ C-তে করা হয়।
- পরে একটু **C++** করা হবে, কার্যত C-র মতো করেই।
- লক্ষ্য হলো **বছরের শেষে** তোমরা নিজেরা একটি কোড লিখতে পারবে, খুব জটিল নয়, কিন্তু **তোমাদের নিজের লেখা**। এখন যা দেখছি সব সাথে থাকবে: শুধু করার মতো জিনিস বাড়বে।

9 সংশোধিত পরীক্ষা ফেরত দেওয়া

সংশোধিত পরীক্ষা **ব্যক্তিগতভাবে** ফেরত দেওয়া হবে: প্রত্যেকে শুধু নিজেরটি পাবে। শিক্ষক ভাবছেন কীভাবে করবেন (ইমেইলে বা Classroom-এর ব্যক্তিগত মন্তব্যে) এবং পাঠানো স্বয়ংক্রিয় করবেন কিনা, যাতে দুজনের পরীক্ষা বদলে যাওয়ার ঝুঁকি না থাকে। পরীক্ষার সাথে দেখে নেওয়ার বিষয়গুলো নিয়ে একটি **মন্তব্যও** থাকতে পারে, যাতে মুখে বলা কথা লিখিতও থাকে। ভাবনাটি হলো শিক্ষক ও ক্লাসের সম্পর্ক যতটা সম্ভব **স্বচ্ছ** ও সম্পূর্ণ হোক।

10 এই পাঠ থেকে যা মনে রাখতে হবে

- nটি বাতি (বিট) দিয়ে **সংমিশ্রণ** হয় 2^nটি।
- `if`: একটি ব্লক করা বা না করা। `if-else`: দুটি ব্লকের একটি বাছা।
- **শর্ত** এমন একটি প্রকাশ যার মান **সত্য** বা **মিথ্যা**, "ইতিবাচক বা নেতিবাচক" নয়।
- `>` সমতা বাদ দেয়, `>=` তা ধরে: 5 ও 5 হলে `a > b` **মিথ্যা**, `a >= b` **সত্য**।
- "অথবা" লেখা হয় `||`, দুটি খাড়া দাগ (**pipe**), slash নয়।
- ছাড়: `if ((eta < 12) || (eta >= 65))`; প্রতিটি শর্তের চারপাশে **সবসময়** **বন্ধনী**।
- দুটি নেস্টেড `if` **কাজ** **করে**, OR বেশি **সংক্ষিপ্ত**।
- C-তে **কার্লি** **বন্ধনী** ব্লক সীমিত করে; **Python**-এ তা করে ইন্ডেন্টেশন।

11 শব্দকোষ: ক্লাসে ব্যবহৃত ইতালীয় শব্দ

- **verifica**: পরীক্ষা
- **sconto / No sconto**: ছাড় / ছাড় নেই
- **età (eta)**: বয়স
- **oppure**: অথবা
- **uguaglianza**: সমতা
- **confronto**: তুলনা
- **espressione**: প্রকাশ
- **indentazione / rientro**: ইন্ডেন্টেশন
- **Ritenta**: আবার চেষ্টা করো
- **portabilità**: বহনযোগ্যতা